

The Celebrity Problem Using Stack

Learn the Elimination Strategy, Not Just the Code

Prism Interview Algorithms Series

Contents

1	The Celebrity Problem Using Stack	4
1.1	Problem: What Are We Trying to Find?	4
1.2	Story: The Company Party	4
1.3	Real-World Analogy	4
1.4	Observation: Two Independent Conditions	5
1.5	Why Naive Thinking Fails	5
1.6	Understanding the Matrix	5
1.7	Valid and Invalid Visual Examples	6
1.7.1	Valid Celebrity	6
1.7.2	Invalid Candidate	6
1.8	Brute-Force Approach	6
1.8.1	Concept Before Code	6
1.8.2	Visualization	7
1.8.3	Brute-Force Pseudo Code	7
1.8.4	Brute-Force Python Code and Driver	7
1.8.5	Console Output	8
1.8.6	Output Explanation	8
1.8.7	Brute-Force Dry Run	8
1.8.8	Complexity	8
1.9	Can We Do Better?	8
1.10	The Elimination Idea	9
1.11	Discovering Stack Naturally	9
1.11.1	Problem	9
1.11.2	Story and Analogy	9
1.11.3	Why Stack Helps	9
1.12	Stack Algorithm	10

1.12.1 Step-by-Step	10
1.12.2 Flowchart	10
1.13 Complete Dry Run with Stack	11
1.13.1 Stack Visualization	11
1.14 Why Verification Is Necessary	11
1.15 Pseudo Code for Stack Solution	12
1.16 Complete Python Implementation	12
1.16.1 Concept Before Code	12
1.16.2 Python Code and Driver Code	12
1.16.3 Console Output	13
1.16.4 Line-by-Line Output Explanation	13
1.16.5 Memory Diagram	14
1.17 Complexity Analysis	14
1.18 Edge Cases	14
1.19 Common Mistakes: 30 Beginner Traps	15
1.20 Interview Thinking	16
1.20.1 Whiteboard Approach	16
1.21 Proof of Correctness	16
1.21.1 Elimination Never Removes the True Celebrity	17
1.21.2 One Remaining Candidate Is the Only Possible Celebrity	17
1.21.3 Verification Guarantees Correctness	17
1.22 Alternative Solutions	17
1.22.1 Two-Pointer Candidate Elimination	17
1.23 Real-World Applications of Elimination	18
1.24 Practice Questions	18
1.25 Summary Cheat Sheet	19
1.25.1 Mind Map	19
1.25.2 Algorithm Summary	19
1.25.3 Final Interview Notes	19

1 The Celebrity Problem Using Stack

1.1 Problem: What Are We Trying to Find?

Problem

There are N people at a party. You are given a function `knows( $A, B$ )`, which returns **True** if person A knows person B , and **False** otherwise. A celebrity is a person who satisfies both rules:

1. Everybody else knows the celebrity.
2. The celebrity knows nobody else.

Return the index of the celebrity if one exists; otherwise return -1 .

The chapter goal is not to memorize a trick. The goal is to understand why one comparison can permanently eliminate one person, why a stack expresses that elimination cleanly, and why a final verification is still required.

1.2 Story: The Company Party

Story

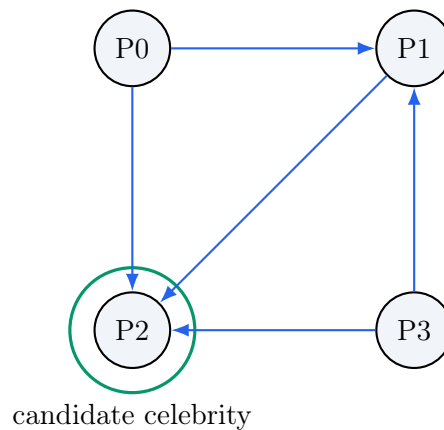
A company hosts a large annual party. Employees, managers, interns, and guests are chatting in small groups. Somewhere in the room may be one special person: a famous investor invited as the guest of honor.

Everybody has heard of this person, so everybody knows them. But the guest of honor came quietly, does not know anyone personally, and is not socializing by name with others.

Your challenge: identify the celebrity without asking every possible question. Each question is expensive: “Does Alice know Bob?” The fewer questions you ask, the better your algorithm.

1.3 Real-World Analogy

Imagine a room of people and a limited question budget. You can ask only yes/no relationship questions. A celebrity is like a sink in a directed graph: arrows flow into them, but no arrows flow out. If P_2 is the celebrity, arrows point from P_0 , P_1 , and P_3 toward P_2 ; P_2 points to nobody.



1.4 Observation: Two Independent Conditions

For person C to be a celebrity:

- **Row condition:** C 's row must contain all zeros, except the diagonal, because the celebrity knows nobody.
- **Column condition:** C 's column must contain all ones, except the diagonal, because everyone knows the celebrity.

These conditions are independent. A person can be known by everyone but still know someone; then they are not a celebrity. A person can know nobody but also be unknown by others; then they are not a celebrity.

1.5 Why Naive Thinking Fails

A beginner often thinks: "For every person, check every other person." This works, but it treats all people as possible celebrities even after evidence proves they cannot be. Interviewers want to see whether you notice eliminations.

Why This Matters

The central interview skill is not stack syntax. It is recognizing that each relationship question $\text{knows}(A, B)$ eliminates either A or B . Once eliminated, a person never needs to be considered as a celebrity again.

1.6 Understanding the Matrix

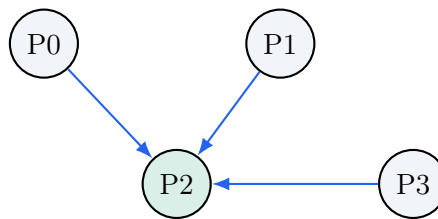
Most versions provide a matrix where $M[i][j] = 1$ means person i knows person j , and $M[i][j] = 0$ means person i does not know person j . Diagonal values $M[i][i]$ are ignored because knowing yourself is irrelevant.

	0	1	2	3	Meaning of row
0	0	1	1	0	P0 knows P1 and P2
1	0	0	1	0	P1 knows P2
2	0	0	0	0	P2 knows nobody
3	0	1	1	0	P3 knows P1 and P2

Column 2 contains ones from rows 0, 1, and 3. Row 2 contains zeros. Therefore P2 is the celebrity.

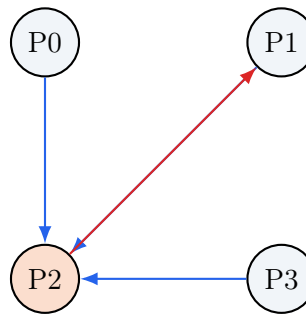
1.7 Valid and Invalid Visual Examples

1.7.1 Valid Celebrity



P2 knows nobody; everyone knows P2.

1.7.2 Invalid Candidate



P2 is known by all, but P2 knows P1, so P2 is not celebrity.

1.8 Brute-Force Approach

1.8.1 Concept Before Code

The direct approach checks every person as a candidate. For a candidate c , scan every person i :

- If c knows i , reject c .
- If i does not know c , reject c .
- Ignore $i = c$, the diagonal case.

1.8.2 Visualization

For candidate P2, inspect row 2 and column 2.

	0	1	2	3
0	0	1	1	0
1	0	0	1	0
2	0	0	0	0
3	0	1	1	0

Blue row: candidate knows nobody.

Green column: everyone knows candidate.

1.8.3 Brute-Force Pseudo Code

```

FOR candidate FROM 0 TO n - 1:
    valid = True
    FOR other FROM 0 TO n - 1:
        IF candidate == other:
            CONTINUE
        IF knows(candidate, other) == True:
            valid = False
        IF knows(other, candidate) == False:
            valid = False
    IF valid == True:
        RETURN candidate
RETURN -1

```

1.8.4 Brute-Force Python Code and Driver

```

1 def find_celebrity_bruteforce(matrix):
2     n = len(matrix)
3
4     for candidate in range(n):
5         valid = True
6
7         for other in range(n):
8             if candidate == other:
9                 continue
10
11             candidate_knows_other = matrix[candidate][other] == 1
12             other_knows_candidate = matrix[other][candidate] == 1
13
14             if candidate_knows_other or not other_knows_candidate:
15                 valid = False
16                 break
17
18         if valid:
19             return candidate
20
21     return -1

```

```

22
23
24 # Driver code
25 party = [
26     [0, 1, 1, 0],
27     [0, 0, 1, 0],
28     [0, 0, 0, 0],
29     [0, 1, 1, 0],
30 ]
31
32 answer = find_celebrity_bruteforce(party)
33 print("Celebrity index:", answer)

```

1.8.5 Console Output

```
Celebrity index: 2
```

1.8.6 Output Explanation

The output is 2 because row 2 contains no outgoing relationships and column 2 contains incoming relationships from all other people. In plain English: P2 knows nobody, and P0, P1, and P3 all know P2.

1.8.7 Brute-Force Dry Run

Candidate	Other	Check	Result
0	1	P0 knows P1, so P0 cannot be celebrity	reject 0
1	0	P0 does not know P1? Actually P0 knows P1, continue; later P1 knows P2	reject 1
2	0	P2 does not know P0 and P0 knows P2	pass
2	1	P2 does not know P1 and P1 knows P2	pass
2	3	P2 does not know P3 and P3 knows P2	pass

1.8.8 Complexity

Brute force may inspect $N - 1$ relationships for each of N candidates, and each inspection may check two matrix cells. Therefore time is $O(N^2)$. Extra space is $O(1)$.

1.9 Can We Do Better?

Ask the interview question out loud: do we really need to compare everyone with everyone? If a single question can prove a person is not the celebrity, why keep checking that person?

Observation

The path to $O(N)$ begins when we stop verifying everyone and start eliminating impossible candidates.

1.10 The Elimination Idea

Take two people, A and B . Ask one question: $\text{knows}(A, B)$.

Question result	Who is eliminated?	Why?
A knows B	A	A celebrity knows nobody, so A cannot be celebrity.
A does not know B	B	Everybody must know the celebrity, so if A does not know B , B cannot be celebrity.

This is the entire trick. Every comparison between two remaining candidates eliminates exactly one of them. The true celebrity, if present, is never eliminated by this rule.

1.11 Discovering Stack Naturally

1.11.1 Problem

We need a simple way to keep a pool of candidates, repeatedly remove two candidates, compare them, discard one, and put the survivor back.

1.11.2 Story and Analogy

Imagine writing names on cards and placing them in a pile. You take two cards from the top, ask one question, tear up one card, and place the survivor back on the pile. Eventually only one card remains.

1.11.3 Why Stack Helps

A stack supports exactly what this process needs:

- **Push:** add a candidate to the pool.
- **Pop:** remove a candidate from the pool.
- **Repeat:** pop two, eliminate one, push one survivor.

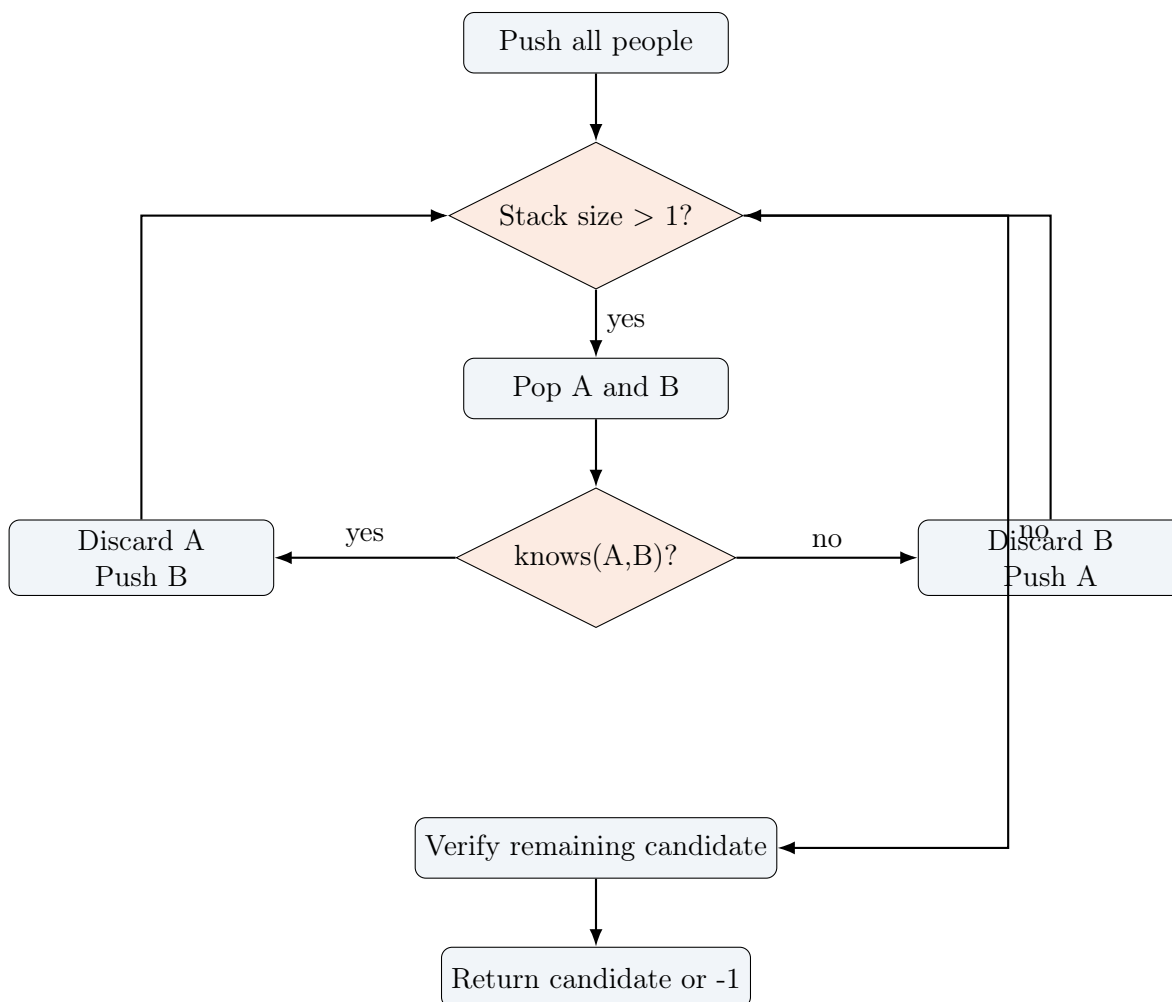
We do not need random access, sorting, or searching. We only need efficient last-in-first-out removal. Python lists provide `append` and `pop` in amortized $O(1)$ time.

1.12 Stack Algorithm

1.12.1 Step-by-Step

1. Push all people 0 to $N - 1$ into a stack.
2. While at least two candidates remain:
 - (a) Pop A and B .
 - (b) If A knows B , discard A , push B .
 - (c) Otherwise discard B , push A .
3. The remaining person is only a possible celebrity.
4. Verify the remaining candidate by checking row and column conditions.

1.12.2 Flowchart



1.13 Complete Dry Run with Stack

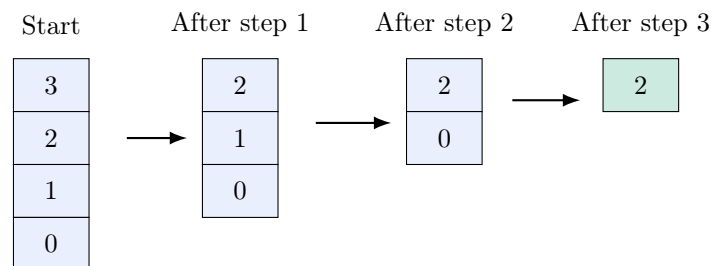
Use the same matrix where P2 is the celebrity.

	0	1	2	3
0	0	1	1	0
1	0	0	1	0
2	0	0	0	0
3	0	1	1	0

Initial stack after pushing 0, 1, 2, 3: bottom [0, 1, 2, 3] top.

Step	Stack before	Pop A	Pop B	Question and elimination	Stack after
1	[0,1,2,3]	3	2	knows(3,2)=True, so 3 cannot be celebrity; push 2	[0,1,2]
2	[0,1,2]	2	1	knows(2,1)=False, so 1 cannot be celebrity; push 2	[0,2]
3	[0,2]	2	0	knows(2,0)=False, so 0 cannot be celebrity; push 2	[2]

1.13.1 Stack Visualization



1.14 Why Verification Is Necessary

The stack phase only proves that everyone except the remaining candidate has been eliminated. It does not prove that the remaining candidate satisfies both celebrity rules.

Warning

Never return the last stack element immediately. Return it only after verifying that its row is all zeros and its column is all ones, ignoring the diagonal.

Counterexample with no celebrity:

	0	1	2
0	0	1	0
1	0	0	1
2	1	0	0

The elimination phase will leave one candidate, but nobody satisfies both conditions. Verification correctly returns -1 .

1.15 Pseudo Code for Stack Solution

```

IF n == 0:
    RETURN -1

stack = empty stack
FOR person FROM 0 TO n - 1:
    PUSH person into stack

WHILE stack size > 1:
    A = POP stack
    B = POP stack

    IF knows(A, B):
        PUSH B # A knows someone, so A is not celebrity
    ELSE:
        PUSH A # A does not know B, so B is not celebrity

candidate = POP stack

FOR person FROM 0 TO n - 1:
    IF person == candidate:
        CONTINUE
    IF knows(candidate, person):
        RETURN -1
    IF NOT knows(person, candidate):
        RETURN -1

RETURN candidate

```

1.16 Complete Python Implementation

1.16.1 Concept Before Code

The code has two phases. Phase one is elimination: repeatedly shrink the candidate pool. Phase two is verification: prove the survivor actually satisfies the row and column rules.

1.16.2 Python Code and Driver Code

```
1 def find_celebrity_stack(matrix):
2     n = len(matrix)
3     if n == 0:
4         return -1
5
6     stack = []
7
8     for person in range(n):
9         stack.append(person)
10
11     while len(stack) > 1:
12         first = stack.pop()
13         second = stack.pop()
14
15         if matrix[first][second] == 1:
16             stack.append(second)
17         else:
18             stack.append(first)
19
20     candidate = stack.pop()
21
22     for person in range(n):
23         if person == candidate:
24             continue
25
26         if matrix[candidate][person] == 1:
27             return -1
28
29         if matrix[person][candidate] == 0:
30             return -1
31
32     return candidate
33
34
35 # Driver code
36 party = [
37     [0, 1, 1, 0],
38     [0, 0, 1, 0],
39     [0, 0, 0, 0],
40     [0, 1, 1, 0],
41 ]
42
43 result = find_celebrity_stack(party)
44 print("Celebrity index:", result)
```

1.16.3 Console Output

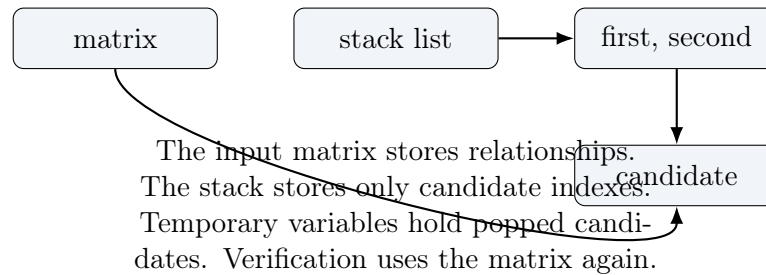
```
Celebrity index: 2
```

1.16.4 Line-by-Line Output Explanation

- Celebrity index: is printed by the driver code.

- 2 is returned by `find_celebrity_stack`.
- The value 2 means person P2 is the only person who knows nobody and is known by every other person.

1.16.5 Memory Diagram



1.17 Complexity Analysis

Approach	Time	Extra Space	Why
Brute force	$O(N^2)$	$O(1)$	For each candidate, scan many people.
Stack elimination	$O(N)$	$O(N)$	Each comparison eliminates one candidate; verification scans once.
Two-pointer elimination	$O(N)$	$O(1)$	Same elimination idea without an explicit stack.

The stack solution performs N pushes, $N - 1$ elimination comparisons, and at most $2(N - 1)$ verification checks. That is linear time.

1.18 Edge Cases

Case	Example	Expected result
No people	<code>[]</code>	-1
One person	<code>[[0]]</code>	Usually 0, because nobody else must know them and they know nobody. Confirm interviewer convention.
Two people with celebrity	<code>[[0, 1], [0, 0]]</code>	1
Two people no celebrity	<code>[[0, 1], [1, 0]]</code>	-1, both know someone.
No celebrity cycle	0 knows 1, 1 knows 2, 2 knows 0	-1

Everyone knows everyone	All off-diagonal values are 1	-1, celebrity knows nobody.
Nobody knows anyone	All values are 0	-1 for $N > 1$, because nobody is known by everyone.
Multiple possible candidates	Impossible for $N > 1$ under strict conditions	Verification prevents false positives.

1.19 Common Mistakes: 30 Beginner Traps

No.	Mistake	Why it happens	Expert habit
1	Returning stack survivor directly	Elimination feels final	Always verify.
2	Reversing <code>knows(a, b)</code> meaning	Matrix row/column confusion	Say “row person knows column person.”
3	Eliminating B when A knows B	Logic swapped	If A knows someone, A is out.
4	Eliminating A when A does not know B	Logic swapped	If A does not know B, B is out.
5	Checking candidate column for zeros	Confuses incoming/outgoing	Column must be ones.
6	Checking candidate row for ones	Confuses celebrity rule	Row must be zeros.
7	Not ignoring diagonal	Self-relationship irrelevant	Skip $i = c$.
8	Treating one person as no celebrity	Convention uncertainty	Ask interviewer; usually return 0.
9	Failing on empty matrix	Assumes $N > 0$	Guard with <code>if n == 0</code> .
10	Using stack but still doing $O(N^2)$ verification	Rechecks all pairs	Verify only survivor.
11	Pushing both candidates back	No elimination occurs	Push exactly one survivor.
12	Pushing eliminated candidate	Careless branch code	Write branch comments.
13	Using <code>pop(0)</code> in Python	Confuses stack with queue	Use <code>pop()</code> .
14	Mutating the matrix	Unnecessary modification	Keep input read-only.
15	Assuming survivor always exists	Stack non-empty only if $N > 0$	Handle $N = 0$.
16	Forgetting column verification	Only checks celebrity knows nobody	Need everyone knows celebrity.
17	Forgetting row verification	Only checks popularity	Need celebrity knows nobody.
18	Checking only direct neighbor	Misunderstands global condition	Verify against every person.
19	Saying stack gives $O(1)$ space	Explicit stack stores all people	Stack version uses $O(N)$ extra space.

20	Not explaining why elimination is safe	Memorized code	Prove each comparison removes non-celebrity.
21	Using nested loops in elimination phase	Brute force hidden inside	One comparison per elimination.
22	Assuming matrix is symmetric	Knowing is directed	$M[i][j]$ may differ from $M[j][i]$.
23	Off-by-one loop bounds	Range mistakes	Use <code>range(n)</code> .
24	Returning boolean instead of index	Misreads prompt	Return candidate index or -1 .
25	Not testing no-celebrity case	Happy-path bias	Include counterexamples.
26	Mislabeling top of stack in dry run	Visual confusion	Write bottom-to-top explicitly.
27	Treating celebrity as most-known only	Popular person may know others	Both conditions required.
28	Using names but coding indexes incorrectly	Mapping errors	Keep index table.
29	Failing to mention API version	Interview may provide <code>knows()</code>	Abstract matrix access behind function.
30	Overcomplicating with graph algorithms	Recognizes directed graph, misses trick	Use elimination, not traversal.

1.20 Interview Thinking

Interview Discussion

A strong interview explanation sounds like this: “A celebrity has zero outgoing edges and $N - 1$ incoming edges. Brute force checks every row and column in $O(N^2)$. To improve, compare two candidates A and B. If A knows B, A cannot be celebrity. If A does not know B, B cannot be celebrity. Thus one question eliminates one person. A stack repeatedly applies this until one candidate remains. Then I verify the candidate because elimination only gives a possible celebrity.”

1.20.1 Whiteboard Approach

1. Write the two celebrity conditions.
2. Draw two people A and B .
3. Derive the two elimination cases.
4. Show stack shrinking from N candidates to one.
5. Emphasize final verification.
6. State complexity clearly.

1.21 Proof of Correctness

1.21.1 Elimination Never Removes the True Celebrity

Consider any comparison between A and B . If A knows B , then A cannot be the true celebrity because the true celebrity knows nobody. If A does not know B , then B cannot be the true celebrity because everyone must know the true celebrity, including A . Therefore the eliminated person is never the true celebrity.

1.21.2 One Remaining Candidate Is the Only Possible Celebrity

The stack loop starts with all people. Every iteration removes exactly one impossible candidate. Since the true celebrity is never removed, if a celebrity exists, they must be the final survivor.

1.21.3 Verification Guarantees Correctness

The final scan checks both required conditions for the survivor. If either condition fails, no celebrity exists. If both pass, the survivor is a valid celebrity. Therefore the algorithm returns the celebrity exactly when one exists.

1.22 Alternative Solutions

1.22.1 Two-Pointer Candidate Elimination

The same elimination idea can be implemented with two pointers or a single candidate variable.

```

1 def find_celebrity_two_pointer(matrix):
2     n = len(matrix)
3     if n == 0:
4         return -1
5
6     candidate = 0
7     for person in range(1, n):
8         if matrix[candidate][person] == 1:
9             candidate = person
10
11    for person in range(n):
12        if person == candidate:
13            continue
14        if matrix[candidate][person] == 1:
15            return -1
16        if matrix[person][candidate] == 0:
17            return -1
18
19    return candidate

```

Method	Strength	Interview note
Brute force	Easiest to derive and verify	Start here to show baseline.
Stack	Clearly models pairwise elimination	Great when topic is stack.

Two pointer

Same time, less space

Mention as optimization after stack.

1.23 Real-World Applications of Elimination

- **Tournament elimination:** one match can remove a competitor from title contention.
- **Candidate filtering:** one failed requirement can remove a candidate from a search pool.
- **Graph problems:** sink/source identification often depends on in-degree and out-degree constraints.
- **Dependency analysis:** impossible nodes can be removed to expose remaining candidates.
- **Leader election:** systems often eliminate invalid leaders before verifying the final leader.
- **Debugging:** binary elimination and hypothesis pruning reduce search space.

1.24 Practice Questions

Practice

Easy

1. Given a matrix, identify whether person 0 is a celebrity.
2. Write the brute-force row and column checks.
3. Explain why diagonal values are ignored.
4. Dry run one stack elimination step for $\text{knows}(3, 2) = 1$.

Medium

1. Implement the stack solution using a `knows(a, b)` helper instead of direct matrix access.
2. Create a no-celebrity matrix where stack elimination leaves candidate 1.
3. Convert the stack solution into the two-pointer solution.
4. Count the maximum number of `knows` calls in elimination plus verification.

Hard

1. Prove that two celebrities cannot exist when $N > 1$.
2. Adapt the solution if `knows(a, b)` is expensive and cached.
3. Design tests for all edge cases in this chapter.
4. Explain the algorithm to a non-technical stakeholder without using matrix terminology.

Debugging

1. Find the bug if a solution returns the survivor without verification.
2. Find the bug if branch `if matrix[a][b] == 1` pushes `a`.
3. Find the bug if verification checks `matrix[person][candidate] == 1` as a failure.

Output Prediction

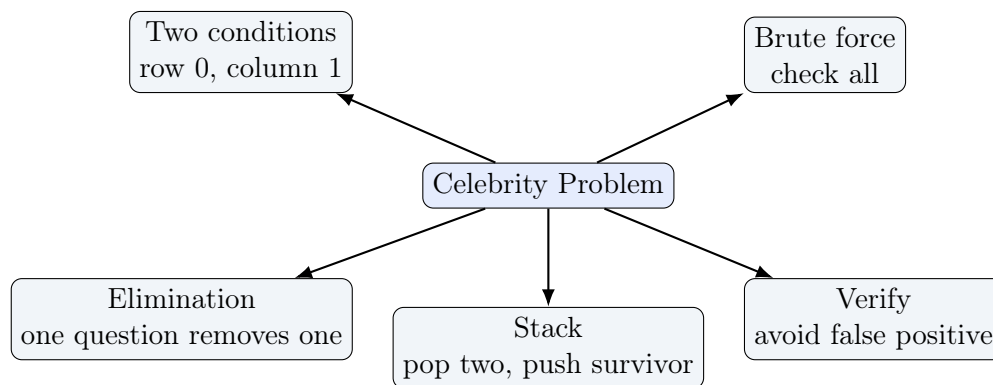
1. Predict output for `[[0, 1], [0, 0]]`.

2. Predict output for $[[0,0,0], [0,0,0], [0,0,0]]$.
3. Predict output for $[[0,1,1], [0,0,1], [0,0,0]]$.

1.25 Summary Cheat Sheet

- Celebrity knows nobody: row is all zeros.
- Everyone knows celebrity: column is all ones.
- Brute force checks every candidate: $O(N^2)$.
- Elimination compares two candidates at a time.
- If A knows B , eliminate A .
- If A does not know B , eliminate B .
- Stack stores current possible candidates.
- Final survivor is only a possible celebrity.
- Verification is mandatory.
- Stack solution: $O(N)$ time, $O(N)$ space.
- Two-pointer solution: $O(N)$ time, $O(1)$ space.
- Never memorize; derive from the two elimination cases.

1.25.1 Mind Map



1.25.2 Algorithm Summary

1. Put all people into a stack.
2. Pop two people and ask whether the first knows the second.
3. Eliminate the person who cannot be celebrity.
4. Push the survivor.
5. Repeat until one candidate remains.
6. Verify row and column conditions.

1.25.3 Final Interview Notes

Lead with brute force, then ask whether every comparison can eliminate someone. Derive the two cases slowly. Use the stack as a natural container for the remaining candidate pool. Close with verification

and complexity. That explanation demonstrates problem solving, not memorization.